

AI Foundations

Stochastic Gradient Descent

Kun Yuan

Peking University

Main contents in this lecture

- Stochastic optimization
- Stochastic gradient descent (SGD)
- Mini-batch SGD
- Momentum SGD

Stochastic optimization

- Consider the stochastic optimization problem:

$$\min_{x \in \mathbb{R}^d} f(x) = \mathbb{E}_{\xi \sim \mathcal{D}}[F(x; \xi)]$$

- ξ is a random variable indicating data samples
 - $\mathcal{D}$ is the data distribution; unknown in advance
 - $F(x; \xi)$ is differentiable in terms of x
- Many applications in signal processing and machine learning

Example: deep neural network

- Recall the DNN training problem

$$\min_{x \in \mathbb{R}^d} \quad \frac{1}{m} \sum_{i=1}^m L(h(x; a_i), b_i)$$

which is a finite-sum problem

- Suppose we have infinite data (a, b) following distribution D , the above problem becomes

$$\min_{x \in \mathbb{R}^d} \quad f(x) = \mathbb{E}_{(a,b) \sim D} L(h(x; a), b)$$

where data pair (a, b) can be regarded as sample ξ .

Stochastic gradient descent

- Recall the problem

$$\min_{x \in \mathbb{R}^d} f(x) = \mathbb{E}_{\xi \sim \mathcal{D}}[F(x; \xi)]$$

- Closed-form of $f(x)$ is unknown; gradient descent is not applicable
- Stochastic gradient descent (SGD):

$$x_{k+1} = x_k - \gamma \nabla F(x_k; \xi_k), \quad \forall k = 0, 1, \dots$$

where ξ_k is a data realization sampled at iteration k .

- Since $\{x_k\}$ are random, all iterates $\{x_k\}$ are also random

Assumption

Let $\mathcal{F}_k = \{x_k, \xi_{k-1}, x_{k-1}, \dots, \xi_0\}$ be the filtration containing all historical variables at and before iteration k (except for ξ_k).

Assumption 1

Given the filtration $\mathcal{F}_k$, we assume

$$\begin{aligned}\mathbb{E}[\nabla F(x_k; \xi_k) | \mathcal{F}_k] &= \nabla f(x_k) \\ \mathbb{E}[\|\nabla F(x_k; \xi_k) - \nabla f(x_k)\|^2 | \mathcal{F}_k] &\leq \sigma^2\end{aligned}$$

Implying **unbiased** stochastic gradient and **bounded** variance.

Convergence: smooth and non-convex scenario

Theorem 1

Suppose $f(x)$ is L -smooth and Assumption 1 holds. If $\gamma \leq 1/L$, SGD will converge at the following rate

$$\frac{1}{K+1} \sum_{k=0}^K \mathbb{E}[\|\nabla f(x_k)\|^2] \leq \frac{2\Delta_0}{\gamma(K+1)} + \gamma L \sigma^2,$$

where $\Delta_0 = f(x_0) - f^*$.

- SGD **cannot** converge to stationary point with constant learning rate
- Smaller learning rate γ or variance σ^2 leads to smaller convergence error

Image Classification

Cifar-10 dataset

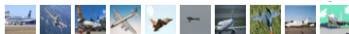
50K training images

10K test images

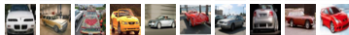
DNN model: ResNet-18

GPU: Tesla V100

airplane



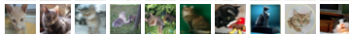
automobile



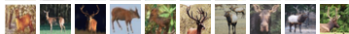
bird



cat



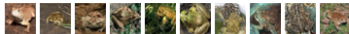
deer



dog



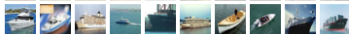
frog



horse



ship

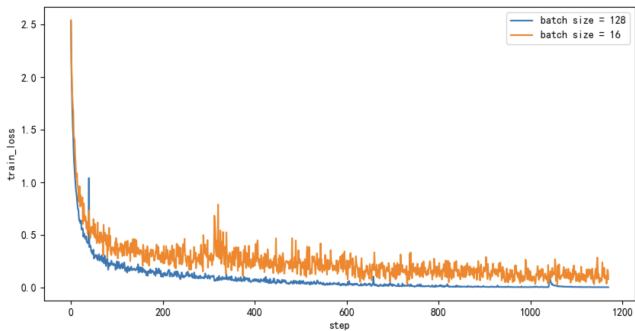


truck



Image Classification

Large batch-size helps training.



Convergence: smooth and non-convex scenario

Corollary 1

Suppose $f(x)$ is L -smooth and Assumption 1 holds. If γ is chosen as

$$\gamma = \left[\left(\frac{2\Delta_0}{(K+1)L\sigma^2} \right)^{-\frac{1}{2}} + L \right]^{-1},$$

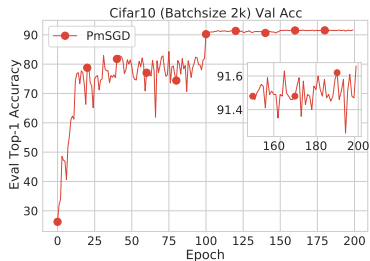
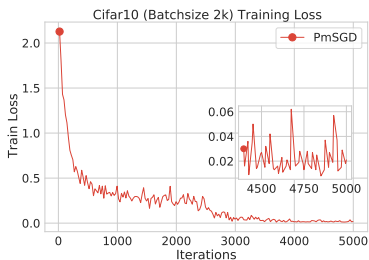
SGD will converge at the following rate

$$\frac{1}{K+1} \sum_{k=0}^K \mathbb{E}[\|\nabla f(x_k)\|^2] \leq \sqrt{\frac{8L\Delta_0\sigma^2}{K+1}} + \frac{2L\Delta_0}{K+1}.$$

where $\Delta_0 = f(x_0) - f^*$.

- Decaying rate leads to exact convergence to stationary point
- When $\sigma^2 = 0$, the above rate **reduces to GD**; rate is tight!
- $O(\sqrt{\sigma^2/K})$ is the dominant rate

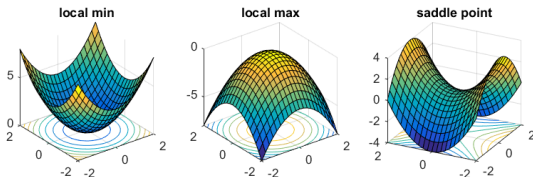
Image Classification



Convergence: smooth and non-convex scenario

$$\frac{1}{K+1} \sum_{k=0}^K \mathbb{E} \|\nabla f(x_k)\|^2 = O \left(\sqrt{\frac{L\sigma^2}{K+1}} + \frac{L}{K+1} \right)$$

- When iteration $K \rightarrow \infty$, it holds that $\mathbb{E} \|\nabla f(x_K)\|^2 \rightarrow 0$
- $\mathbb{E} \|\nabla f(x_K)\|^2 \rightarrow 0$ implies SGD converges to a stationary solution
- A stationary solution can be local min, local max, or saddle point¹



¹Image source: from Prof. Rong Ge's online post

Convergence: smooth and non-convex scenario

- Generally speaking, approaching the stationary solution is the best result we can get for SGD; no guarantee to approach the global minimum
- Empirically, SGD performs extremely well when training DNN
- Recent advanced studies show SGD can escape local maximum, saddle point, and even “sharp” local minimum, see, e.g., (Ge et al., 2015; Sun et al., 2015; Jin et al., 2017; Du et al., 2018, 2019; Kleinberg et al., 2018)
- SGD can even find global minimum under certain conditions, e.g. the PL condition (Karimi et al., 2016)

However, we will skip these interesting results in this lecture

Convergence: smooth and convex scenario

Theorem 2

Suppose $f(x)$ is convex and L -smooth. Under Assumption 1, if $\gamma \leq 1/(2L)$, SGD will converge at the following rate

$$\frac{1}{K+1} \sum_{k=0}^K \mathbb{E}[f(x_k) - f(x^*)] \leq \frac{\Delta_0}{\gamma(K+1)} + \gamma\sigma^2$$

where $\Delta_0 = \|x_0 - x^*\|^2$. If we further choose $\gamma = \left[\left(\frac{\Delta_0}{(K+1)\sigma^2} \right)^{-\frac{1}{2}} + 2L \right]^{-1}$, SGD converges as follows

$$\frac{1}{K+1} \sum_{k=0}^K \mathbb{E}[f(x_k) - f(x^*)] \leq 2\sqrt{\frac{\sigma^2 \Delta_0}{K+1}} + \frac{2L\Delta_0}{K+1}.$$

Tight rate. Reduces to GD when $\sigma^2 = 0$.

Convergence: smooth and strongly-convex scenario

Theorem 3

Suppose $f(x)$ is μ -strongly convex and L -smooth. Under Assumption 1, if $\gamma \leq 1/L$, SGD will converge at the following rate

$$\mathbb{E}[f(x_k)] - f^* \leq (1 - \gamma\mu)^k \Delta_0 + \frac{\gamma L \sigma^2}{\mu}.$$

where $\Delta_0 = f(x_0) - f^$. If we further choose $\gamma = \min\{\frac{1}{L}, \frac{1}{\mu K} \ln\left(\frac{\mu^2 \Delta_0 K}{L \sigma^2}\right)\}$, SGD will converge at the following rate*

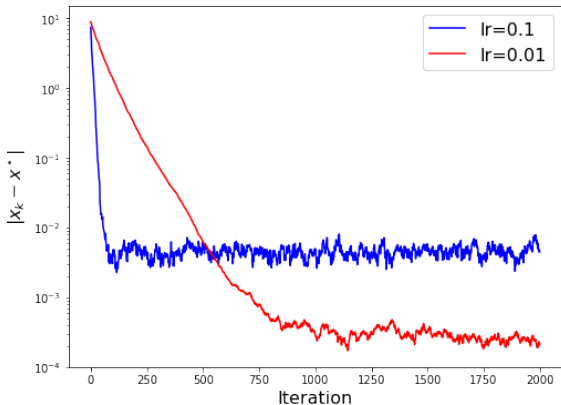
$$\mathbb{E}[f(x_K)] - f^* = \tilde{O}\left(\frac{L \sigma^2}{\mu^2 K} + \Delta_0 \exp\left(-\frac{\mu}{L} K\right)\right)$$

where the $\tilde{O}(\cdot)$ notation hides all logarithm terms.

Tight rate. Reduces to GD when $\sigma^2 = 0$.

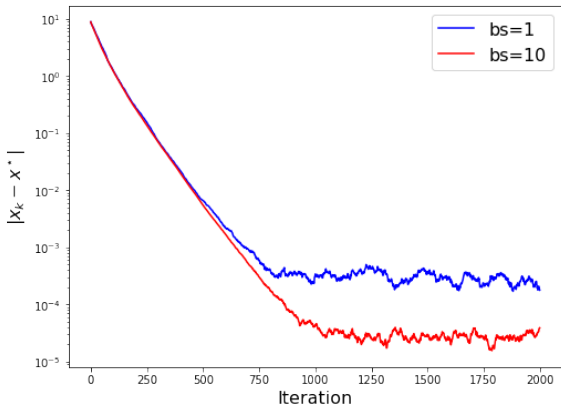
Convergence: smooth and strongly-convex scenario

Linear regression: $\min \frac{1}{2N} \sum_{i=1}^N (a_i^T x - b_i)^2$



Convergence: smooth and strongly-convex scenario

Linear regression: $\min \frac{1}{2N} \sum_{i=1}^N (a_i^T x - b_i)^2$



SGD summary on rate and complexity

Algorithm	Scenario	Rate	Complexity
SGD	non-convex	$\frac{\sigma}{\sqrt{K}} + \frac{1}{K}$	$\frac{\sigma^2}{\epsilon^2} + \frac{1}{\epsilon}$
	generally-convex	$\frac{\sigma}{\sqrt{K}} + \frac{1}{K}$	$\frac{\sigma^2}{\epsilon^2} + \frac{1}{\epsilon}$
	strongly-convex	$\frac{\sigma^2}{K} + \exp(-K)$	$\frac{\sigma^2}{\epsilon} + \ln(\frac{1}{\epsilon})$
GD	non-convex	$\frac{1}{K}$	$\frac{1}{\epsilon}$
	generally-convex	$\frac{1}{K}$	$\frac{1}{\epsilon}$
	strongly-convex	$\exp(-K)$	$\ln(\frac{1}{\epsilon})$

- SGD recovers GD when $\sigma^2 = 0$
- Existence of σ^2 deteriorates the convergence rate significantly

SGD with mini-batch

- In DNN, it is common to sample a batch of data to estimate gradient
- Mini-batch SGD iterate as follows

$$g_k = \frac{1}{B} \sum_{b=1}^B \nabla F(x_k; \xi_k^{(b)}),$$

$$x_{k+1} = x_k - \gamma g_k$$

where B is the batch-size.

- B samples together can provide a much better estimate of $\nabla f(x)$

SGD with mini-batch

We first introduce the filtration

$$\mathcal{F}_k^B = \{x_k, \{\xi_{k-1}^{(b)}\}_{b=1}^B, x_{k-1}, \{\xi_{k-2}^{(b)}\}_{b=1}^B, \dots, x_0\}$$

Assumption 2

Given the filtration $\mathcal{F}_k^B$, we assume

$$\begin{aligned}\mathbb{E}[\nabla F(x_k; \xi_k^{(b)}) | \mathcal{F}_k^B] &= \nabla f(x_k), \\ \mathbb{E}[\|\nabla F(x_k; \xi_k^{(b)}) - \nabla f(x_k)\|^2 | \mathcal{F}_k^B] &\leq \sigma^2.\end{aligned}$$

Moreover, we assume $\{\xi_k^{(b)}\}_{b=1}^B$ are independent of each other for any k .

Implying that mini-batch can provide a much better estimate of $\nabla f(x)$

$$\mathbb{E}[\|g_k - \nabla f(x_k)\|^2 | \mathcal{F}_k^B] = \frac{1}{B^2} \sum_{b=1}^B \mathbb{E}[\|\nabla F(x_k; \xi_k^{(b)}) - \nabla f(x_k)\|^2 | \mathcal{F}_k^B] \leq \frac{\sigma^2}{B}$$

Mini-batch SGD convergence

Theorem 4

Suppose $f(x)$ is L -smooth and Assumption 2 holds. Mini-batch SGD will converge at the following rate

$$\frac{1}{K+1} \sum_{k=0}^K \mathbb{E}[\|\nabla f(x_k)\|^2] = O\left(\sqrt{\frac{L\Delta_0\sigma^2}{B(K+1)}} + \frac{L\Delta_0}{K+1}\right)$$

where $\Delta_0 = f(x_0) - f^*$.

Large batch-size **accelerates** the convergence; $B = 1$ reduces to SGD

Similar results also hold in convex and strongly-convex scenarios.

Mini-batch SGD convergence

Comparison in the dominant sample complexity

Large batch-size can significantly reduce the sample complexity

Convexity	SGD	Mini-batch SGD
Non-convex	$\frac{L}{\epsilon^2}$	$\frac{L}{B\epsilon^2}$
Convex	$\frac{L}{\epsilon^2}$	$\frac{L}{B\epsilon^2}$
Strongly convex	$\frac{L}{\mu\epsilon}$	$\frac{L}{\mu B\epsilon}$

Gradient descent can be slow

- Gradient descent can be very slow for ill-conditioned problems
- For example, GD converges very slow when μ/L is sufficiently small²

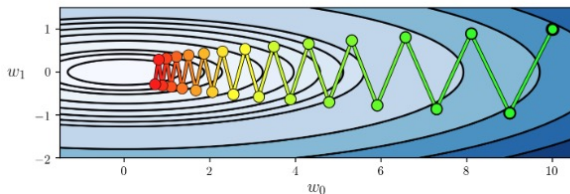


Figure: GD converges slow for ill-conditioned problem

²Image is from https://github.com/jermwatt/machine_learning_refined

Gradient descent with Polyak's momentum

- We have to alleviate the “Zig-Zag” to accelerate the algorithm
- **Polyak's momentum** method, a.k.a, **heavy-ball** gradient method

$$x_k = x_{k-1} - \gamma \nabla f(x_{k-1}) + \beta(x_{k-1} - x_{k-2})$$

where $\beta \in (0, 1)$ is the momentum parameter

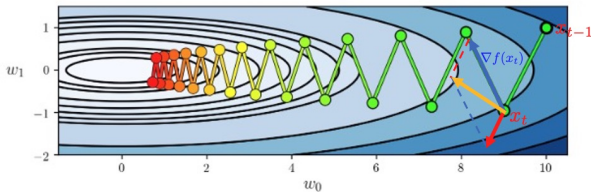


Figure: Momentum can alleviate the “Zig-Zag”

Gradient descent with Polyak's momentum

- We have to alleviate the “Zig-Zag” to accelerate the algorithm
- **Polyak's momentum** method, a.k.a, **heavy-ball** gradient method

$$x_k = x_{k-1} - \gamma \nabla f(x_{k-1}) + \beta(x_{k-1} - x_{k-2})$$

where $\beta \in (0, 1)$ is the momentum parameter

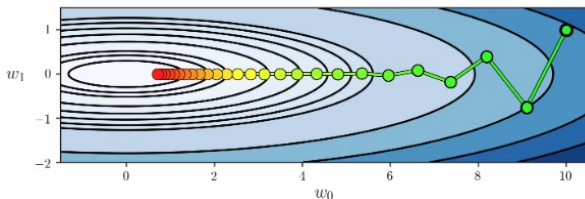


Figure: Momentum can alleviate the “Zig-Zag”

Boris Polyak



Boris Polyak (1935—2023)

Russian mathematician

Polyak's momentum; stochastic gradient descent; stochastic averaging

Gradient descent with Polyak's momentum

$$\min_x f(x) = \frac{1}{2}x^\top Ax \quad \text{where} \quad A = \begin{bmatrix} 20 & 0 \\ 0 & 1 \end{bmatrix}$$

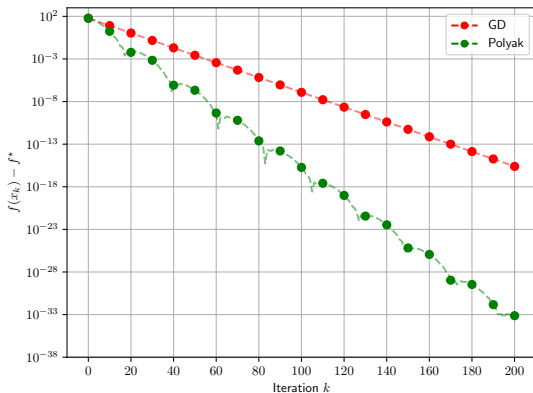


Figure: GD v.s. Polyak momentum

Gradient descent with Nesterov's momentum

- Gradient descent with **Nesterov's momentum**, a.k.a, **Nesterov accelerated gradient (NAG)** method

$$y_{k-1} = x_{k-1} + \beta(x_{k-1} - x_{k-2})$$

$$x_k = y_{k-1} - \gamma \nabla f(y_{k-1})$$

where $\beta \in (0, 1)$ is the momentum parameter

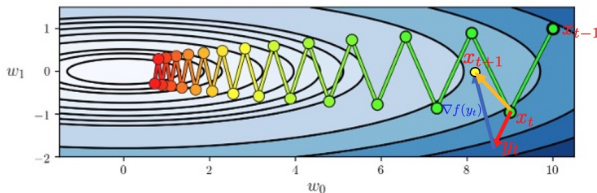


Figure: Nesterov method can alleviate the “Zig-Zag”

Yurii Nesterov



Yurii Nesterov (1956—)

Russian mathematician

Boris Polyak's student

Nesterov acceleration

Gradient descent with Nesterov's momentum

$$\min_x f(x) = \frac{1}{2}x^\top Ax \quad \text{where} \quad A = \begin{bmatrix} 20 & 0 \\ 0 & 1 \end{bmatrix}$$

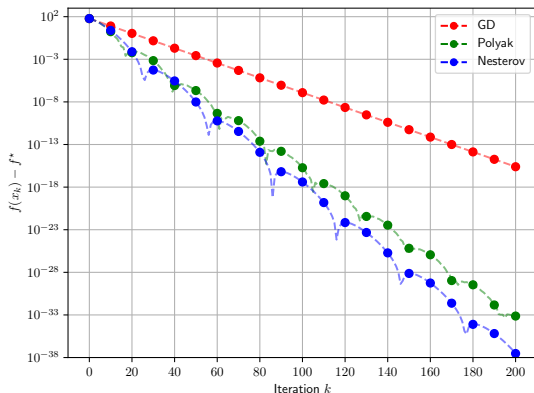


Figure: GD v.s. various momentums

Momentum SGD

- Recall the stochastic optimization

$$\min_{x \in \mathbb{R}^d} \mathbb{E}_{\xi \in \mathcal{D}} [F(x; \xi)]$$

- The standard SGD algorithm is

$$x_{k+1} = x_k - \gamma \nabla F(x_k; \xi_k)$$

- Momentum SGD

$$x_{k+1} = x_k - \gamma \nabla F(x_k; \xi_k) + \beta(x_k - x_{k-1}) \quad (2)$$

where $\beta \in [0, 1)$ is the momentum coefficient

Momentum SGD

- Momentum SGD can be rewritten into

$$\begin{aligned}m_k &= \beta m_{k-1} + \nabla F(x_k; \xi_k) \\ x_{k+1} &= x_k - \gamma m_k\end{aligned}$$

where $m_0 = 0$. This is how PyTorch implement it³.

- To see it, we have the following recursion from the second line

$$\beta x_k = \beta x_{k-1} - \gamma \beta m_{k-1}.$$

Subtract it from the second line, we have

$$\begin{aligned}x_{k+1} - \beta x_k &= x_k - \beta x_{k-1} - \gamma(m_k - \beta m_{k-1}) \\ &= x_k - \beta x_{k-1} - \gamma \nabla F(x_k; \xi_k).\end{aligned}$$

Regrouping the terms, we achieve Momentum SGD recursion (2).

³<https://pytorch.org/docs/stable/generated/torch.optim.SGD.html>

References I

- R. Ge, F. Huang, C. Jin, and Y. Yuan, “Escaping from saddle points—online stochastic gradient for tensor decomposition,” in *Conference on learning theory*. PMLR, 2015, pp. 797–842.
- J. Sun, Q. Qu, and J. Wright, “When are nonconvex problems not scary?” *arXiv preprint arXiv:1510.06096*, 2015.
- C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan, “How to escape saddle points efficiently,” in *International Conference on Machine Learning*. PMLR, 2017, pp. 1724–1732.
- S. S. Du, X. Zhai, B. Póczos, and A. Singh, “Gradient descent provably optimizes over-parameterized neural networks,” *arXiv preprint arXiv:1810.02054*, 2018.
- S. Du, J. Lee, H. Li, L. Wang, and X. Zhai, “Gradient descent finds global minima of deep neural networks,” in *International Conference on Machine Learning*. PMLR, 2019, pp. 1675–1685.

References II

- B. Kleinberg, Y. Li, and Y. Yuan, “An alternative view: When does sgd escape local minima?” in *International Conference on Machine Learning*. PMLR, 2018, pp. 2698–2707.
- H. Karimi, J. Nutini, and M. Schmidt, “Linear convergence of gradient and proximal-gradient methods under the polyak-łojasiewicz condition,” in *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2016, pp. 795–811.